

Antonie Aureliu Valentin

TODO 1 - **build_finger_table**

Am implementat functia, folosind explicatiile din enunt si comentarii.

Printr-un for (int i = 1; i <= 8; i *= 2), am calculat pozitiile teoretice. Am „hardcodat” valoarea $8 = (2^{M-1})$, deoarece M este fixat (4), astfel fiind mereu 4 pozitii. Pentru fiecare pozitie am calculat succesorul folosind functia deja implementata **find_successor_simple**.

TODO 2 - **closest_preceding_finger**

Am parcurs descrescator pozitiile, gasind primul nod care se gaseste in intervalul circular (self.id, K). Deoarece functia deja implementata **in_interval** ne spune daca x se afla in intervalul (a, b], am folosit **in_interval(self.finger[i].node, self.id, key - 1)**. Pentru cazul particular cand key = 0, pentru ca key - 1 sa nu fie < 0 (functia nu da corect pentru valori negative), am asignat key = 16 $\Leftrightarrow$ key - 1 = 15.

TODO 3 - **handle_lookup_request**

Am adaugat self.id in path, am crescut lungimea path-ului (msg->path_len).

Am modificat msg->current_id (= self.id).

Am verificat daca succesorul este responsabil de key (folosind **in_interval(msg->key, self.id, self.successor)**).

In caz afirmativ, am adaugat succesorul in path, am crescut lungimea path-ului si am trimis TAG_LOOKUP_REP catre initiator, extragand rank-ul initiatorului folosind functia deja implementata **rank_from_id**.

```
MPI_Send(msg, sizeof(LookupMsg), MPI_BYTE, rank_from_id(msg->initiator_id), TAG_LOOKUP_REP, MPI_COMM_WORLD)
```

Altfel, am asignat next = closest_preceding_finger(msg->key), conform comentariilor si enuntului si am trimis TAG_LOOKUP_REQ catre rank-ul „next-ului”

```
MPI_Send(msg, sizeof(LookupMsg), MPI_BYTE, rank_from_id(next), TAG_LOOKUP_REQ, MPI_COMM_WORLD)
```

TODO 4

Pentru fiecare key din fisier, am alocat mesajul (structura), apoi l-am construit si, mai apoi, l-am trimis folosind tag-ul TAG_LOOKUP_REQ catre propriul rank (**world_rank**).

TODO 5

Am initializat 2 variabile: nr (numarul de lookup-uri procesate, egal cu 0 initial) si left (numarul de mesaje cu TAG_DONE care mai trebuie primite, egal cu world_size - 1 initial).

In caz in care nr_lookups = 0, trimit TAG_DONE catre toti imediat

Altfel (bucula principala):

Primesc mesaje folosind MPI_Recv:

```
MPI_Recv(msg, sizeof(LookupMsg), MPI_BYTE, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
```

Daca primesc mesaj cu **TAG_LOOKUP_REQ**, apelez **handle_lookup_request(msg)**

Daca primesc mesaj cu **TAG_LOOKUP_REP**, am terminat lookup-ul si adaug path-ul intr-un buffer. Fac acest lucru, deoarece afisarea trebuie sa se realizeze respectand ordinea cheilor din fisier. Incrementez nr, si, cand nr = nr_lookups, inseamna ca toate lookup-urile initiale local au fost rezolvate si pot sa trimit TAG_DONE catre celelalte noduri.

Altfel (primesc mesaj cu **TAG_DONE**), decrementez left. Cand left = 0 si nr = nr_lookups, ies din while (break).

La final, fac afisarea din buffer si dezaoloc toate structurile.